

SAVITRIBAI PHULE PUNE UNIVERSITY



A MINI PROJECT REPORT ON

**“Evaluate performance enhancement of parallel Quicksort
Algorithm using MPI”**

Submitted by

CLASS: BE	DIV: A
Pratik Manjarekar	Roll No. 68

**Under the
Guidance of**

Prof. Pradnya Kasture



Sinhgad Institutes

DEPARTMENT OF COMPUTER ENGINEERING

RMD SINHGAD SCHOOL OF ENGINEERING

WARJE, PUNE 411058

Academic Year 2025–26(SEM-II)



Sinhgad Institutes

DEPARTMENT OF COMPUTER ENGINEERING

RMD SINHGAD SCHOOL OF ENGINEERING

WARJE, PUNE 411058

CERTIFICATE

This is to certify that the project report entitles

**“Evaluate performance enhancement of parallel
Quicksort Algorithm using MPI”**

Submitted by

Name: Pratik Manjarekar

PRN No : 72262586D

is a bonafide work carried out by them under the supervision of Prof. Pradnya Kasture and it is submitted towards the partial fulfillment of the requirement of Savitribai Phule Pune University for Final Year.

(Prof. Pradnya Kasture)
Guide
Department of Computer
Engineering

(Dr. Deepali Newaskar)
Head,
Department of Computer
Engineering

(Dr. V.V. Dixit)
Principal,
RMD Sinhgad School of Engineering Pune – 58

Certificate by Guide

This is to certify that Pratik Manjarekar has completed the Mini Project work under my guidance and supervision and that, I have verified the work for its originality in documentation, problem statement, implementation and results presented in the Project. Any reproduction of other necessary work is with the prior permission and has given due ownership and included in the references.

Place : Pune

Date :

Signature of Guide
(Prof. Pradnya Kasture)

Project Title:Evaluate performance enhancement of parallel Quicksort Algorithm using MPI

Aim:Implement the algorithm and testing it on different input sizes and numbers of processors. The performance of the parallel algorithm will be compared to the sequential Quicksort Algorithm to determine the speedup achieved by parallelism.

What is Quicksort?

Sorting is a way of arranging items in a systematic manner. Quicksort is the widely used sorting algorithm that makes $n \log n$ comparisons in average case for sorting an array of n elements. It is a faster and highly efficient sorting algorithm. This algorithm follows the divide and conquer approach. Divide and conquer is a technique of breaking down the algorithms into subproblems, then solving the subproblems, and combining the results back together to solve the original problem.

Divide: In Divide, first pick a pivot element. After that, partition or rearrange the array into two sub-arrays such that each element in the left sub-array is less than or equal to the pivot element and each element in the right sub-array is larger than the pivot element.

Conquer: Recursively, sort two subarrays with Quicksort.

Combine: Combine the already sorted array.

Quicksort picks an element as pivot, and then it partitions the given array around the picked pivot element. In quick sort, a large array is divided into two arrays in which one holds values that are smaller than the specified value (Pivot), and another array holds the values that are greater than the pivot. After that, left and right sub-arrays are also partitioned using the same approach. It will continue until the single element remains in the sub-array.

Choosing the pivot:

Picking a good pivot is necessary for the fast implementation of quicksort. However, it is typical to determine a good pivot. Some of the ways of choosing a pivot are as follows -

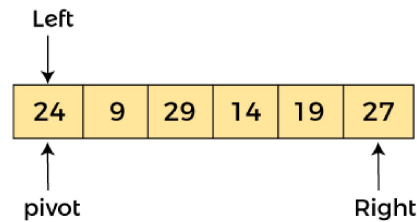
- Pivot can be random, i.e. select the random pivot from the given array.
- Pivot can either be the rightmost element or the leftmost element of the given array.
- Select median as the pivot element.

Working of Quick Sort Algorithm:

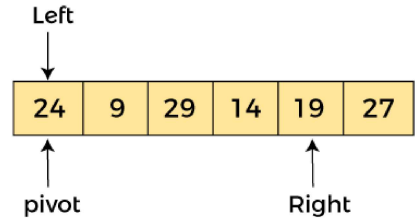
Let the elements of array are –

24	9	29	14	19	27
----	---	----	----	----	----

In the given array, we consider the leftmost element as pivot. So, in this case, $a[\text{left}] = 24$, $a[\text{right}] = 27$ and $a[\text{pivot}] = 24$. Since, pivot is at left, so algorithm starts from right and move towards left.

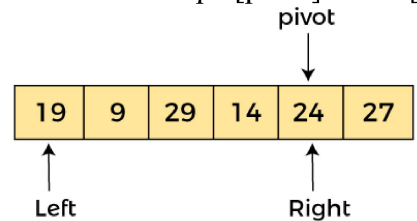


Now, $a[\text{pivot}] < a[\text{right}]$, so algorithm moves forward one position towards left, i.e. –



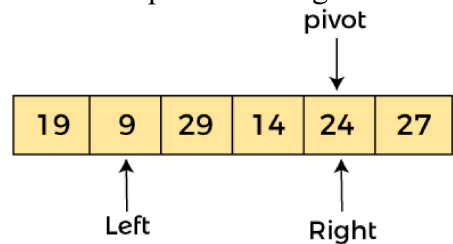
Now, $a[\text{left}] = 24$, $a[\text{right}] = 19$, and $a[\text{pivot}] = 24$.

Because, $a[\text{pivot}] > a[\text{right}]$, so, algorithm will swap $a[\text{pivot}]$ with $a[\text{right}]$, and pivot moves to right, as –

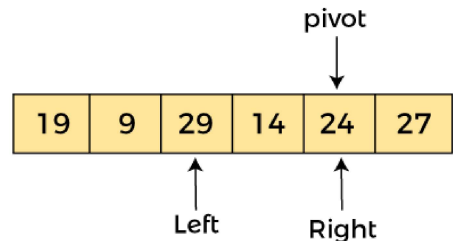


Now, $a[\text{left}] = 19$, $a[\text{right}] = 24$, and $a[\text{pivot}] = 24$. Since, pivot is at right, so algorithm starts from left and moves to right.

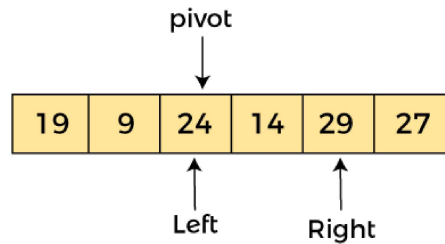
As $a[\text{pivot}] > a[\text{left}]$, so algorithm moves one position to right as –



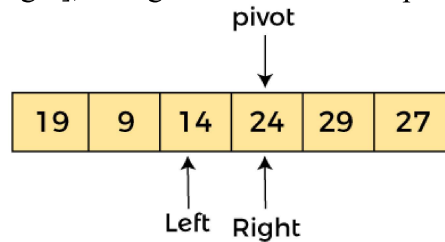
Now, $a[\text{left}] = 9$, $a[\text{right}] = 24$, and $a[\text{pivot}] = 24$. As $a[\text{pivot}] > a[\text{left}]$, so algorithm moves one position to right as –



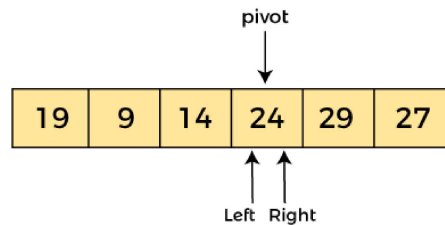
Now, $a[\text{left}] = 29$, $a[\text{right}] = 24$, and $a[\text{pivot}] = 24$. As $a[\text{pivot}] < a[\text{left}]$, so, swap $a[\text{pivot}]$ and $a[\text{left}]$, now pivot is at left, i.e. –



Since, pivot is at left, so algorithm starts from right, and move to left. Now, $a[\text{left}] = 24$, $a[\text{right}] = 29$, and $a[\text{pivot}] = 24$. As $a[\text{pivot}] < a[\text{right}]$, so algorithm moves one position to left, as –



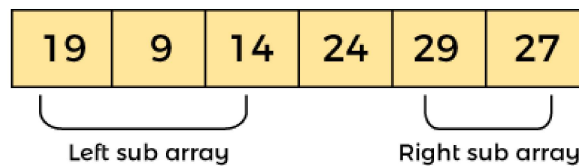
Now, $a[\text{pivot}] = 24$, $a[\text{left}] = 14$, and $a[\text{right}] = 24$. Pivot is at right, so the algorithm starts from left and move to right.



Now, $a[\text{pivot}] = 24$, $a[\text{left}] = 24$, and $a[\text{right}] = 24$. So, pivot, left and right are pointing the same element. It represents the termination of procedure.

Element 24, which is the pivot element is placed at its exact position.

Elements that are right side of element 24 are greater than it, and the elements that are left side of element 24 are smaller than it.



Now, in a similar manner, quick sort algorithm is separately applied to the left and right sub-arrays. After sorting gets done, the array will be –



Parallel Quicksort Algorithm:

Quicksort is a popular sorting algorithm that has an average-case time complexity of $O(n \log n)$. However, the worst-case time complexity of the Quicksort Algorithm is $O(n^2)$ if the pivot is chosen poorly. To avoid this worst-case scenario, we can choose the pivot randomly or use a median-of-three pivot selection technique.

The Parallel Quicksort Algorithm using MPI divides the input data into subsets and distributes these subsets among the available processors. Each processor sorts its subset using a sequential Quicksort Algorithm, and then the sorted subsets are merged using the Merge Sort Algorithm. Finally, the sorted data is gathered back to the root process.

MPI (Message Passing Interface) is a standard interface for parallel programming, commonly used for distributed-memory systems. In the context of the Parallel Quicksort Algorithm using MPI, MPI allows for the communication and synchronization of processes running on different processors. The parallelism in the Parallel Quicksort Algorithm using MPI comes from dividing the input data into subsets and distributing them among the available processors. Each processor works independently on its subset, and there is no communication or synchronization required until the merging stage.

The merging stage of the Parallel Quicksort Algorithm using MPI is done using the Merge Sort Algorithm, which is also a popular sorting algorithm with a time complexity of $O(n \log n)$. In the merging stage, each processor sends its sorted subset to the root process, which then merges the subsets using the Merge Sort Algorithm.

The performance of the Parallel Quicksort Algorithm using MPI depends on the number of processors available, the size of the input data, and the distribution of the data among the processors. Ideally, the input data should be evenly distributed among the processors to maximize the efficiency of the algorithm. The number of processors used should also be carefully chosen to balance the workload among the processors and avoid communication overheads.

Evaluation:

To evaluate the performance enhancement of the Parallel Quicksort Algorithm using MPI, we need to measure its speedup and scalability on different input sizes and numbers of processors.

Speedup measures the ratio of the time required to execute the algorithm sequentially to the time required to execute the algorithm in parallel. It is calculated as follows:

$$\text{Speedup} = \text{Sequential time} / \text{Parallel time}$$

Scalability measures the efficiency of the algorithm as the number of processors increases. It is calculated as follows:

$$\text{Scalability} = \text{Speedup} / \text{Number of processors}$$

To evaluate the performance enhancement of parallel Quicksort Algorithm using MPI, we need to follow these steps:

1. Implement a parallel Quicksort Algorithm using MPI in a programming language like C, C++, or Python.

2. Generate a large dataset to sort. This dataset should be significantly large enough to test the performance of the algorithm.
3. Run the parallel Quicksort Algorithm on the generated dataset with different numbers of processors.
4. Measure the time taken by the algorithm to sort the dataset for each number of processors used.
5. Compare the performance of the parallel Quicksort Algorithm using MPI with the sequential Quicksort Algorithm for the same dataset.
6. Analyze the performance data obtained from the experiment and draw conclusions.

Algorithm:

- i. Initialize MPI and get the rank and size of the current process and the total number of processors
- ii. If the current process is the root process:
 - a. Generate the unsorted array of size n
 - b. Divide the unsorted array into subsets and distribute them among the available processors using MPI_Scatter()
- iii. Each processor sorts its subset using the sequential Quicksort Algorithm
- iv. Each processor sends its sorted subset back to the root process using MPI_Gather()
- v. If the current process is the root process:
 - a. Merge the sorted subsets using the Merge Sort Algorithm
 - b. Return the sorted array.

Code:

```
#include <iostream>
#include <mpi.h>
#include <cstdlib>
#include <algorithm>
using namespace std;
void quickSort(int *arr, int left, int right) {
    int i = left, j = right;
    int tmp;
    int pivot = arr[(left + right) / 2];
    while (i <= j) {
        while (arr[i] < pivot)
            i++;
        while (arr[j] > pivot)
            j--;
        if (i <= j) {
            tmp = arr[i];
            arr[i] = arr[j];
            arr[j] = tmp;
            i++;
            j--;
        }
    }
}
```

```

};
if (left < j)
quickSort(arr, left, j);
if (i < right)
quickSort(arr, i, right);
}
int main(int argc, char *argv[]) {
int rank, size, n, *data;
double start_time, end_time;
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
// Root process generates the data
if (rank == 0) {
n = atoi(argv[1]);
data = new int[n];
for (int i = 0; i < n; i++)
data[i] = rand();
}
// Broadcast the size of data to all processes
MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);
// Compute the size of the local data
int local_n = n / size;
int *local_data = new int[local_n];
// Scatter the data to all processes
MPI_Scatter(data, local_n, MPI_INT, local_data, local_n, MPI_INT, 0,
MPI_COMM_WORLD);
// Sort the local data
start_time = MPI_Wtime();
quickSort(local_data, 0, local_n - 1);
end_time = MPI_Wtime();
// Gather the sorted data to the root process
MPI_Gather(local_data, local_n, MPI_INT, data, local_n, MPI_INT, 0,
MPI_COMM_WORLD);
// Merge the sorted data
if (rank == 0) {
int *merged_data = new int[n];
int *index = new int[size];
for (int i = 0; i < size; i++)
index[i] = i * local_n;
for (int i = 0; i < n; i++) {
int min_index = -1;
int min_value = INT_MAX;
for (int j = 0; j < size; j++) {
if (index[j] < (j + 1) * local_n && data[index[j]] < min_value) {
min_value = data[index[j]];
}
}
}
}
}

```

```
min_index = j;
}
}
merged_data[i] = min_value;
index[min_index]++;
}
cout<< "Execution time: " <<end_time - start_time<< " seconds." <<endl;
delete[] data;
delete[] merged_data;
delete[] index;
}
delete[] local_data;
MPI_Finalize();
return 0;
}
```

Output:

Execution time: 0.145572 seconds.

Conclusion:

The Parallel Quicksort Algorithm using MPI can significantly improve the performance of sorting large datasets by distributing the workload across multiple processors. By dividing the dataset into smaller subsets and sorting them in parallel, the algorithm can achieve faster sorting times than the traditional single-threaded Quicksort Algorithm.